

AWS Identity and Access Management (IAM)

Introduction

AWS Identity and Access Management (IAM) is a web service that helps you securely control access to AWS resources. You use IAM to control who is authenticated (signed in) and authorized (has permissions) to use AWS resources.

Components of IAM

1. Users

Using IAM, we can create and manage AWS users and use permissions to allow and deny their access to AWS resources.

2. Groups

Groups are collections of users and have policies attached to them. A group allows you to manage permissions for multiple users at once.

3. Roles

An IAM role defines a set of permissions for making AWS service requests. Trusted entities such as IAM users, applications, or AWS services can assume a role.

4. Policies

Policies assign permissions to a user, group, role, or resource. They are written in JSON format and define allowed or denied actions.

Types of Policies

1. Service Control Policies (SCPs)

SCPs are used in AWS Organizations to manage permissions across multiple AWS accounts. They do not grant permissions but limit the maximum permissions that can be granted.

2. Identity-Based Policies

These policies are attached to IAM users, groups, or roles. They define what actions an identity can perform on which resources under specific conditions.

3. Resource-Based Policies

These policies are attached to specific AWS resources and define which principal (user, role, or service) is allowed or denied actions on the resource.

1. Service Control Policy (SCP)

Applied at organization/account level – like a company-wide rule.

Story:

You are the **manager of a company (AWS Organization)**.

You tell your team:

"Nobody in any AWS account in our organization is allowed to delete S3 buckets – ever!"

Even if someone tries to give themselves that permission, AWS will **block it**.

Purpose:

- Limit the **maximum** permissions across AWS accounts
- Doesn't **give** permission, only **blocks/limits**

Real Example:

```
Json
{
  "Effect": "Deny",
  "Action": "s3:DeleteBucket",
  "Resource": "*"
}
```

This means:

No one in the organization can delete any S3 bucket – even if their IAM policy allows it.

2. Identity-Based Policy

Attached to a person or app (IAM User, Group, or Role)

Story:

You hire a **developer in Account A**, and you say:

"You are allowed to upload and view images in the **S3 bucket for the photo app**."

So, the permission is given **to the developer's IAM identity**.

Purpose:

- Grants permission **to a person or role**
- Specifies **what actions** they can take on **which resources**

Real Example:

json

```
{  
  
  "Effect": "Allow",  
  
  "Action": ["s3:GetObject", "s3:PutObject"],  
  
  "Resource": "arn:aws:s3:::photo-app-bucket/*"  
}
```

This means:

The developer can upload and download images in the photo-app-bucket.

3. Resource-Based Policy

Attached to the AWS resource itself (like an S3 bucket)

Story:

You are the owner of the **S3 bucket in Account A**.

You want to let your **partner in Account B** access your bucket to **download images**.

Instead of giving permissions to the partner's IAM user, you write a permission policy **on the S3 bucket itself** saying:

"My partner (Account B) can read my images."

Purpose:

- Gives access to **someone else** from **another AWS account**
- Permission is written **on the resource itself**

Real Example:

json

```
{  
  
  "Effect": "Allow",
```

```
"Principal": {  
  
  "AWS": "arn:aws:iam::222222222222:user/PartnerUser"  
  
},  
  
"Action": "s3:GetObject",  
  
"Resource": "arn:aws:s3:::photo-app-bucket/*"  
  
}
```

This means:

IAM user PartnerUser from Account B can **read files** from photo-app-bucket.

Summary Table

Policy Type	Where It's Attached	Controls Who Can...	Example Purpose
Service Control Policy	AWS Organization / Account	Anyone in org	"No one can delete S3 buckets in the whole org"
Identity-Based Policy	IAM User / Role / Group	That specific identity	"Developer can upload photos to S3"
Resource-Based Policy	AWS Resource (e.g. S3)	Anyone specified in policy	"Allow Partner in Account B to view images in my bucket"

Example: EC2 to Access S3

To allow EC2 to access S3 resources:

1. Create an IAM role with an S3 policy.
2. Attach the role to the EC2 instance.

Common S3 CLI Commands

List S3 buckets:

```
aws s3 ls
```

Copy file from EC2 to S3:

```
aws s3 cp sourcefile s3://s3-bucket-name
```

Download file from S3 to EC2:

```
aws s3 cp s3://s3-bucket-name/filename .
```